

Last four weeks: computational characterization of regular languages.

i.e.: What kind of computers can recognize regular languages?

Next two classes: Syntactic characterization of regular languages.

i.e.: What are simple rules for generating regular languages?

Regular Expressions

Idea: “Represent a language with a string of symbols”

Need two things:

- Rules for valid strings of symbols.
- Rules for interpreting the strings as languages.

A valid string of symbols will be called a *regular expression*.

We’ll start with atomic regular expressions, and then build them up using combination rules.

Atomic regular expressions:

- $\emptyset$ is a regular expression.

$$L(\emptyset) = \emptyset.$$

- ε is a regular expression.

$$L(\varepsilon) = \{\varepsilon\}$$

- Given finite alphabet Σ and $a \in \Sigma$, a is a regular expression.

$$L(a) = \{a\}.$$

Rules for combining regular expressions into larger expressions:

- Concatenation
- Union
- Kleene star

Compound regular expressions:

- Let α and β be regular expressions.

$\alpha\beta$ is a regular expression.

$$L(\alpha\beta) = L(\alpha)L(\beta)$$

Give a regular expression for the language $\{aabb\}$.

- Let α and β be regular expressions.

$\alpha + \beta$ is a regular expression.

$$L(\alpha + \beta) = L(\alpha) \cup L(\beta)$$

What is $L(0011 + 010)$?

Parentheses work the way you would expect.

Concat before $+$.

What is $L(00(01 + 11))$?

What is $L((1 + 0)(11 + 00))$?

- Let α be a regular expression.

α^* is a regular expression.

$$L(\alpha^*) = L(\alpha)^*$$

What is $L((0 + 1)^*)$?

What is $L((0 + 1)^*1)$?

What is $L((00)^* + (000)^*)$?

What is $L((00)^*(000)^*)$?

Order of operations:

1. Kleene star
2. Concatenation
3. Union

Actual uses of regular expressions: pattern matching.

Theorem: Let A be a language. There is a regular expression α such that $L(\alpha)$ if and only if A is regular.

Today: Given a regular expression α , show that $L(\alpha)$ is regular.

Structural induction.

- Give an NFA for $L(\emptyset)$.
- Give an NFA for $L(\varepsilon)$.
- Given $a \in \Sigma$, give an NFA for $L(a)$.

Let α, β be regular expressions. Assume $L(\alpha)$ and $L(\beta)$ are regular.

- Is $L(\alpha\beta)$ regular? (How do you take NFAs for $L(\alpha)$ and $L(\beta)$ and make an NFA for $L(\alpha\beta)$?)
- Is $L(\alpha + \beta)$ regular? (How do you take NFAs for $L(\alpha)$ and $L(\beta)$ and make an NFA for $L(\alpha + \beta)$?)
- Is $L(\alpha^*)$ regular? (How do you take an NFA for α^* and make an NFA for $L(\alpha^*)$?)

Hence, for every regular expression α , $L(\alpha)$ is regular.

Simplification Rules

What is the language of this regular expression?

- $(a + b)^*b$
- $(a^*b)^*a^*b$

Same language. Which is easier to look at?

Some rules we've already established. Let $\alpha \equiv \beta$ mean $L(\alpha) = L(\beta)$.

- $\alpha + (\beta + \gamma) \equiv (\alpha + \beta) + \gamma$
- $\alpha + \beta \equiv \beta + \alpha$
- $\alpha(\beta\gamma) \equiv (\alpha\beta)\gamma$
- $\alpha(\beta + \gamma) \equiv \alpha\beta + \alpha\gamma$
- $(\alpha + \beta)\gamma \equiv \alpha\gamma + \beta\gamma$

Which of these rules are true?

- $\alpha + \alpha \equiv \alpha$
- $\alpha\beta \equiv \beta\alpha$ (false)
- $\varepsilon + \alpha\alpha^* \equiv \alpha^*$
- (Also true: $\varepsilon + \alpha^*\alpha \equiv \alpha^*$)

Which of these are true?

- $\alpha + \emptyset \equiv \alpha$
- $\varepsilon\alpha \equiv \alpha\varepsilon \equiv \alpha$
- $\emptyset\alpha \equiv \alpha\emptyset \equiv \emptyset$
- $\alpha + \varepsilon = \alpha$ (false)

Fancy rules for deriving new rules.

- $\beta + \alpha\gamma + \gamma \equiv \gamma \Rightarrow \alpha^*\beta + \gamma \equiv \gamma$
- $\beta + \gamma\alpha + \gamma \equiv \gamma \Rightarrow \beta\alpha^* + \gamma \equiv \gamma$

Note: The book states the last two as

- $L(\beta + \alpha\gamma) \subseteq L(\gamma) \Rightarrow L(\alpha^*\beta) \subseteq L(\gamma)$
- $L(\beta + \gamma\alpha) \subseteq L(\gamma) \Rightarrow L(\beta\alpha^*) \subseteq L(\gamma)$

As an exercise, convince yourself that this is equivalent.

Some things you can derive using these last two:

- $(\alpha\beta)^* \equiv \alpha(\beta\alpha)^*$

- $(\alpha^*\beta)^*\alpha^* \equiv (\alpha + \beta)^*$
- $\alpha^*(\beta\alpha^*)^* \equiv (\alpha + \beta)^*$
- $(\varepsilon + \alpha)^* \equiv \alpha^*$
- $\alpha\alpha^* \equiv \alpha^*\alpha$.

Why this is nice:

- Actually practical for using regular expressions.
- Can do algebra with regular expressions, and therefore regular languages.

All properties of regular languages we've considered can be proved algebraically!